

Understanding Syntax: Python and R Compared

A Comparative Introduction for Data Analysis

Data Analysis for Decision-Making

2026-04-28

Why Compare Python and R?

“The goal is not to choose a language — it is to understand data. (Manus Ai)”

- Both **R** and **Python** are industry-standard tools for data analysis and statistical computing
- **R** was designed specifically for statistical analysis and data visualisation
- **Python** is a general-purpose language widely adopted in data science, machine learning, and automation
- This presentation focuses on **syntax** — the grammar of each language — to help you read, write, and translate code confidently

Python and R at a Glance

Feature	R	Python
Primary purpose	Statistical computing & graphics	General-purpose programming
Key strength	Statistics, visualisation (ggplot2)	ML, automation, versatility
Main data tool	data.frame / tibble	pandas DataFrame
Package ecosystem	tidyverse, ggplot2	pandas, scikit-learn
Index start	1 (one-based)	0 (zero-based)

Shared Foundations: What Python and R Have in Common

- Both are **interpreted** languages
- Both support **vectorised operations**
- Both integrate with **Quarto / R Markdown / Jupyter** for reproducible reporting
- Both support **functional programming**
- Both use the **data frame** as the primary tabular data structure

R — Vectorised Operation

```
1 scores <- c(85, 90, 78, 92)
2 scaled <- scores / 100
3 print(scaled)
4 # [1] 0.85 0.90 0.78 0.92
```

Python — Vectorised Operation

```
1 import numpy as np
2 scores = np.array([85, 90, 78, 92])
3 scaled = scores / 100
4 print(scaled)
5 # [0.85 0.90 0.78 0.92]
```


Assignment and Variables: Different Conventions, Same Logic

R — Assignment

```
1 # Primary assignment operator: <-
2 student_name <- "Srikumar"
3 score       <- 92.5
4 is_passed   <- TRUE
5
6 # Print by typing the name
7 student_name
8 # [1] "Srikumar"
```

- Uses `<-` (preferred) or `=`
- Prints by simply typing the variable name
- Dots allowed in names: `my.var`

Python — Assignment

```
1 # Only assignment operator: =
2 student_name = "Srikumar"
3 score       = 92.5
4 is_passed    = True
5
6 # Explicit print required in scripts
7 print(student_name)
```

- Uses `=` exclusively
- Requires `print()` in scripts
- Underscores standard: `my_var`

Data Structures: Vectors, Lists, and Data Frames

R – Data Structures

```
1 # Vector: same-type elements
2 grades <- c(85, 90, 78)
3
4 # Named list: mixed types
5 student <- list(
6   name = "Srikumar",
7   grade = 90
8 )
9
10 # Data frame: tabular data
11 df <- data.frame(
12   name = c("Alice", "Bob"),
13   score = c(88, 76)
14 )
```

Python – Data Structures

```
1 import pandas as pd
2
3 # List: ordered, mixed types
4 grades = [85, 90, 78]
5
6 # Dictionary: key-value pairs
7 student = {
8   "name": "Srikumar",
9   "grade": 90
}
```



```
10 }  
11  
12 # DataFrame: tabular data  
13 df = pd.DataFrame({  
14     "name": ["Alice", "Bob"],  
15     "score": [88, 76]  
16 })
```

Note: In R, data frames are built on vectors, while in Python, DataFrames are provided through the pandas library.

Indexing and Filtering: 1-Based vs. 0-Based

R — Indexing & Filtering

```
1 grades <- c(85, 90, 78, 92)
2
3 # 1-based: first element is [1]
4 grades[1]      # 85
5 grades[2:3]    # 90 78
6
7 df <- data.frame(
8   name = c("Alice", "Bob", "Carol"),
9   score = c(88, 76, 95)
10 )
11
12 # Filter rows where score > 80
13 df[df$score > 80, ]
```

Python — Indexing & Filtering

```
1 import pandas as pd
2
3 grades = [85, 90, 78, 92]
4
5 # 0-based: first element is [0]
6 grades[0]      # 85
7 grades[1:3]    # [90, 78]
8
9 df = pd.DataFrame({
10   "name": ["Alice", "Bob", "Carol"],
```

```
11     "score": [88, 76, 95]
12 })
13
14 # Filter rows where score > 80
15 df[df["score"] > 80]
```

Key rule: R starts counting at **1**; Python starts at **0**. Slicing in Python excludes the end index.

Translating R into Python: A Complete Workflow

R — Student Score Analysis

```
1 scores <- c(72, 85, 90, 88,  
2           76, 95, 60, 83)  
3  
4 mean_score <- mean(scores)  
5 sd_score   <- sd(scores)  
6  
7 cat("Mean:", mean_score,  
8     "\nSD:", sd_score, "\n")  
9  
10 # Filter: above average  
11 above_avg <- scores[scores > mean_score]  
12 print(above_avg)
```

Python — Equivalent Code

```
1 import numpy as np  
2  
3 scores = np.array([72, 85, 90, 88,  
4                  76, 95, 60, 83])  
5  
6 mean_score = np.mean(scores)  
7 sd_score   = np.std(scores, ddof=1)  
8  
9 print(f"Mean: {mean_score:.2f}"  
10       f"\nSD: {sd_score:.2f}")  
11
```

```
12 # Filter: above average
13 above_avg = scores[scores > mean_score]
14 print(above_avg)
```

Translation map:

R	Python
<code>mean()</code>	<code>np.mean()</code>
<code>sd()</code>	<code>np.std(ddof=1)</code>
<code>cat()</code>	<code>print(f"...")</code>
<code>scores[condition]</code>	<code>scores[condition]</code>

R in Action: Exploratory Data Analysis with dplyr

Scenario: Analyse MBA student performance by programme using R's **dplyr** pipeline.

```
1 library(dplyr)
2
3 # Sample dataset
4 students <- data.frame(
5   name    = c("Alice", "Bob", "Carol", "David", "Eva"),
6   score    = c(88, 72, 95, 65, 83),
7   program = c("Finance", "Marketing", "Finance", "HR", "Marketing")
8 )
9
10 # Analysis pipeline using the pipe operator |>
11 summary_stats <- students |>
12   filter(score >= 70) |>           # keep students who passed
13   group_by(program) |>           # group by programme
14   summarise(
15     avg_score = mean(score),      # calculate mean score
16     n_students = n()             # count students
17   ) |>
18   arrange(desc(avg_score))       # sort by highest average
19
20 print(summary_stats)
```

Key dplyr verbs: **filter()** → **select()** → **mutate()** → **summarise()** → **arrange()**

The pipe operator **|>** reads like a sentence: *“Take students, then filter, then group,*

| *then summarise.”*

Exercise: Translate R Code into Python

Task: Translate the following R code into Python using [pandas](#).

```
1 # R Code – Translate this into Python
2 library(dplyr)
3
4 sales <- data.frame(
5   product = c("A", "B", "C", "D"),
6   revenue = c(15000, 8500, 22000, 11000),
7   units   = c(120, 85, 200, 95)
8 )
9
10 result <- sales |>
11   mutate(price_per_unit = revenue / units) |>
12   filter(revenue > 10000) |>
13   arrange(desc(revenue))
14
15 print(result)
```

Hints:

R (dplyr)

Python (pandas)

`mutate(col = expr)` `.assign(col = expr)`

`filter(condition)` `.query("condition")`

R (dplyr)

Python (pandas)

`arrange(desc(col))` `.sort_values("col", ascending=False)`

Exercise: Model Solution

Python Solution:

```
1 import pandas as pd
2
3 sales = pd.DataFrame({
4     "product": ["A", "B", "C", "D"],
5     "revenue": [15000, 8500, 22000, 11000],
6     "units":   [120, 85, 200, 95]
7 })
8
9 result = (
10     sales
11     .assign(price_per_unit = sales["revenue"] / sales["units"])
12     .query("revenue > 10000")
13     .sort_values("revenue", ascending=False)
14     .reset_index(drop=True)
15 )
16
17 print(result)
```

Key observations:

- `mutate()` → `.assign()` — adds a new computed column
- `filter()` → `.query()` — filters rows by condition
- `arrange(desc())` → `.sort_values(ascending=False)` — sorts descending

- Method chaining in pandas mirrors the pipe operator in R

Key Takeaways

Concept comparison at a glance:

Concept	R	Python
Assignment	<code>x <- 5</code>	<code>x = 5</code>
First element	<code>v[1]</code>	<code>v[0]</code>
Filter rows	<code>df[df\$col > x,]</code>	<code>df[df["col"] > x]</code>
Mean	<code>mean(v)</code>	<code>np.mean(v)</code>
Group & summarise	<code>group_by() > summarise()</code>	<code>df.groupby().agg()</code>
Pipe / chain	<code> ></code> or <code>%>%</code>	<code>.method()</code> chaining
Add column	<code>mutate()</code>	<code>.assign()</code>

Core takeaways:

- R and Python share the same **analytical logic** — only the syntax differs
- R's **1-based indexing** and `<-` assignment are the key adjustment points

- The `dplyr` pipeline in R mirrors **pandas method chaining** in Python
- For this course: **R is the primary language**; Python adds professional versatility
- Mastering one language makes learning the other significantly easier

Quiz Time:

Scan to Join the Game!

Quiz: Test Your Syntax Knowledge!

The Dataset:

ID	Sex	Age	Weight	Calories	Sport
1	F	21	48	1700	60
2	F	19	55	1800	120
3	F	23	50	2300	180
4	M	18	71	2000	60
5	M	20	77	2800	240
6	M	61	85	2500	30

```
1 # R Code to create this dataset
2 df <- data.frame(
3   ID = c(1, 2, 3, 4, 5, 6),
4   Sex = c("f", "f", "f", "m", "m", "m"),
5   Age = c(21, 19, 23, 18, 20, 61),
6   Weight = c(48, 55, 50, 71, 77, 85),
7   Calories = c(1700, 1800, 2300, 2000, 2800, 2500),
8   Sport = c(60, 120, 180, 60, 240, 30)
9 )
```


Question 1: Filtering

R Code: `df %>% filter(sport > 100)`

What is the Python equivalent?

- A. `df.filter(sport > 100)`
 - B. `df[df["sport"] > 100]`
 - C. `df.select("sport" > 100)`
 - D. `df["sport" > 100]`
-

Question 2: Adding Columns

R Code: `df %>% mutate(ratio = calories / weight)`

What is the Python equivalent?

- A. `df["ratio"] = calories / weight`
 - B. `df["ratio"] = df["calories"] / df["weight"]`
 - C. `df.mutate(ratio = calories / weight)`
 - D. `df.add_column("ratio")`
-

Quiz Question 3: Grouping & Aggregating

R Code: `df %>% group_by(sex) %>% summarise(avg = mean(calories))`

What is the Python equivalent?

- A. `df.groupby("sex").mean(calories)`
 - B. `df.groupby("sex")["calories"].mean()`
 - C. `df.mean(group="sex")`
 - D. `group(df, "sex")`
-

Quiz Question 4: Sorting

R Code: `df %>% arrange(desc(weight))`

What is the Python equivalent?

- A. `df.sort(weight)`
 - B. `df.sort_values("weight", ascending=False)`
 - C. `df.order(weight)`
 - D. `df.sort(desc(weight))`
-

Quiz Question 5: Selecting Columns

R Code: `df %>% select(age, weight, calories)`

What is the Python equivalent?

- A. `df[["age", "weight", "calories"]]`

- B. `df.select(age, weight, calories)`
- C. `df["age", "weight", "calories"]`
- D. `df.columns("age", "weight", "calories")`

Conclusion and References

“Learn the concepts deeply in R – then map them to Python as needed.(Manus Ai)”

Conclusion:

- Both R and Python are powerful, **complementary** tools for data-driven decision-making
- Understanding syntax differences reduces friction when reading documentation or collaborating
- R remains the preferred language for **statistical rigour** and academic reporting in this course
- Python’s versatility makes it a valuable addition to any data analyst’s toolkit

References:

- Wickham, H. & Grolemund, G. (2017). *R for Data Science*. O’Reilly. r4ds.had.co.nz
- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O’Reilly Media.

- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. r-project.org
- Python Software Foundation (2024). *Python Language Reference*. docs.python.org
- Quarto (2024). *Revealjs Presentations*. quarto.org



Data Analysis for Decision-Making | Srikumar Ravichandran

Speaker notes